Lab	Specific Program	Bloom's Level
1	Design an algorithm to find the **second largest element** in an array using the minimum number of comparisons. Compare it with the sorting-based approach by counting comparisons and execution time.	L4
2	Develop a program that experimentally verifies the growth rates of **O(log n), O(n), O(n log n), O(n^2), and O(2^n)** by generating execution-time graphs for increasing input sizes.	L5
3	Implement **recursive and iterative Binary Search**. Compare execution time, number of comparisons, recursion depth, and memory usage for sorted datasets of different sizes.	L4
4	Implement recursive and iterative versions of **Tower of Hanoi**. Analyze the recurrence relation, number of moves, recursion tree, and execution time.	L4
5	Modify **Merge Sort** to count inversions in an array. Compare the running time with the brute-force inversion counting algorithm.	L5
6	Implement **Quick Sort** using four pivot-selection strategies (first, last, middle, random). Compare performance for sorted, reverse-sorted, and random datasets.	L5
7	Compare **Merge Sort, Quick Sort, and Heap Sort** on datasets containing random numbers, duplicate values, and partially sorted data. Justify the most suitable algorithm for each dataset.	L5
8	Implement **Karatsuba multiplication** for very large integers and compare execution time with the conventional multiplication algorithm.	L5
9	Implement **Strassen's Matrix Multiplication** and compare the number of arithmetic operations and execution time with conventional matrix multiplication for matrices of different sizes.	L5
10	Implement **Binary Tree Traversals (Recursive and Iterative)**. Compare stack usage, execution time, and traversal correctness on balanced and skewed trees.	L4
11	Implement **Prim's and Kruskal's algorithms** on sparse and dense graphs. Compare total execution time and justify which algorithm is preferable for each graph type.	L5
12	Implement **Dijkstra's Algorithm** using (i) adjacency matrix and (ii) priority queue (min-heap). Compare performance for graphs with different densities.	L5
13	Solve the **Fractional Knapsack Problem** using the Greedy method. Compare the obtained solution with the optimal solution of the 0/1 Knapsack problem for the same dataset.	L4
14	Compute the **Binomial Coefficient C(n,k)** using recursion, Dynamic Programming (tabulation), and memoization. Compare execution time and memory usage.	L5

15	Implement **Warshall's Algorithm** for transitive closure and verify graph connectivity for directed graphs of varying sizes. Compare results with DFS/BFS-based reachability.	L4
16	Implement **Floyd's Algorithm** for all-pairs shortest paths. Compare its performance with repeated execution of Dijkstra's algorithm on weighted graphs.	L5
17	Construct an **Optimal Binary Search Tree** from given successful and unsuccessful search probabilities. Compare average search cost with a standard Binary Search Tree.	L5
18	Solve the **0/1 Knapsack Problem** using recursion, Dynamic Programming, and memoization. Compare execution time, memory consumption, and solution quality.	L5
19	Solve the **N-Queens Problem** using Backtracking. Measure the number of recursive calls, backtracking steps, and execution time for N = 4 to N = 12.	L4
20	Solve the **Subset Sum Problem** using Backtracking and Dynamic Programming. Compare the number of explored states, execution time, and memory utilization.	L5
21	Solve the **Assignment Problem** using Branch and Bound. Compare the number of nodes explored with the exhaustive search approach.	L5
22	Solve the **Traveling Salesman Problem** using Branch and Bound. Compare execution time and explored search space with brute-force enumeration.	L5
23	Solve the **0/1 Knapsack Problem** using Branch and Bound and compare its performance with Dynamic Programming and Greedy approaches for different input sizes.	L5
24	Develop a program to experimentally classify selected algorithms into **Polynomial Time and Exponential Time** by measuring execution time for increasing input sizes and plotting the results.	L4
25	Implement an **Approximation Algorithm for the Traveling Salesman Problem (Nearest Neighbor)** and compare its tour cost and execution time with the optimal Branch and Bound solution. Compute the approximation ratio.	L5